CanItShip: Closing the Feedback Loop on Agent Tooling via Trajectory Analysis and Agentic DevX Metrics

Evgenii Kniazev
Databricks
evgenii.kniazev@databricks.com

Arseny Kravchenko
Databricks
arseny.kravchenko@databricks.com

Fabian Jakobs
Databricks
fabian.jakobs@databricks.com

Igor Rekun
Databricks
igor.rekun@databricks.com

Abstract

When AI agents fail to build production applications, should we improve the agent or its environment? Recent work suggests that environment quality (templates, tools, guidance) matters more than model selection [11]. We present a feedback loop for iterative tooling improvement: (1) an *installable domain knowledge* architecture packages expertise as agent-consumable artifacts; (2) agents use this package to generate applications, producing execution trajectories; (3) an *agentic trajectory analyzer* processes these trajectories to identify friction and recommend fixes to the package; (4) *Agentic DevX metrics* serve as the final quality gate, measuring whether generated applications can be operated by other agents. We validate on Databricks data applications across two agent backends (Claude Code and Opencode+Sonnet). On 50 data-focused prompts requiring SQL, schema access, and platform integration, we achieve 100% build and type safety, 98% runtime success, and 96% local runability at $1.61/app—compared to 73.3% viability on easier consumer prompts in prior work. Agentic evaluation reveals that even apps passing scripted checks fail under autonomous operation (73% agentic runability, 36% deployability). The trajectory analyzer closes the loop: it processes failure traces from agentic evaluation, recommends fixes, which are applied to the domain package and applications, and improvements are measured by rerunning the same evaluations. The architecture targets artifacts amenable to automatic optimization [1, 9], potentially closing the loop without human intervention.

CCS Concepts

• **Software and its engineering → Software testing and debugging**; *Software development process management*.

Keywords

CanItShip, agent-agnostic tooling, agentic code generation, trajectory analysis, developer experience metrics, feedback loop

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
EASE '26, Glasgow, United Kingdom
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/26/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Evgenii Kniazev, Arseny Kravchenko, Fabian Jakobs, and Igor Rekun. 2026. CanItShip: Closing the Feedback Loop on Agent Tooling via Trajectory Analysis and Agentic DevX Metrics. In *Proceedings of the International Conference on Evaluation and Assessment in Software Engineering (EASE '26), June 9–12, 2026, Glasgow, United Kingdom.* ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

AI agents can generate functional software, but they lack domain-specific knowledge required for production applications. The conventional response is building specialized agents. We take a different approach: improve what agents have access to.

This approach is motivated by recent findings [11]: when comparing agent performance across environments while holding the model constant, environment quality (templates, tools, guidance) had larger effects than model upgrades. An agent with excellent tooling outperforms a better model with poor tooling.

Accepting that tooling matters raises a question: how do we improve it systematically? Manual inspection doesn't scale. End-state metrics (build pass/fail) don't reveal causes. Early work on automated code assessment [10] showed that comprehensible source code metrics can classify changes effectively; we extend this intuition to agent-generated applications. We need a feedback loop that: (1) captures how agents actually use tooling; (2) identifies friction points and root causes; (3) produces actionable recommendations; (4) verifies improvements against agent-centric criteria.

We instantiate each component:

(1) **Installable Domain Knowledge** (Section 2). An architecture for packaging domain expertise as agent-consumable artifacts, exposing tools via CLI commands compatible with emerging skill-based standards.
(2) **Agentic Trajectory Analyzer** (Section 3). A two-phase system: parallel friction extraction with a fast model followed by agentic synthesis with a reasoning model that has source code access.
(3) **CanItShip: Agentic DevX Metrics** (Section 4). The final quality gate: eight metrics across two evaluation modes—shell-scripted checks (**Ls**) for deterministic properties and agentic evaluation (**Ra**) for autonomous operation and deployment readiness.

Validation. We validate across Claude Code (primary, n=47) and Opencode+Sonnet (n=35) on Databricks data applications. On

47 Claude Code applications: 100% build and type safety, 98% runtime success, and 96% local runability at $1.61/app average. The trajectory analyzer identified improvements that we implemented and verified through multiple iterations.

2 Installable Domain Knowledge

Developers already use capable agents (Claude Code, Opencode). Rather than asking them to adopt yet another tool, we bring domain knowledge to the agents they already use. A user installs our package into their existing environment; the agent gains domain expertise without workflow changes.

The package has three components: context layers that inject domain knowledge progressively, tools exposed via CLI, and a state machine that enforces validation before deployment.

Context layers. Agents have limited context windows. Dumping all domain knowledge upfront wastes tokens and can confuse the model. Instead, we inject context progressively: tool definitions are always available at the protocol level, workflow patterns activate on first discovery call, target-specific constraints load when the application type is detected, and SDK-specific templates inject after scaffolding.

CLI tools and MCP parity. We expose domain functionality through CLI commands, a pattern that Cloudflare [6] and Anthropic [3] found effective. Each CLI command maps 1:1 to a Model Context Protocol (MCP) tool definition, ensuring the same domain knowledge is accessible to any MCP-compliant agent. Lifecycle commands (`scaffold`, `validate`, `deploy`) and data exploration commands (schema discovery, batch queries) are available as both CLI tools and MCP skills, providing the domain-specific capabilities on top of standard workspace tools.

State machine. Applications cannot deploy unless they pass validation after their most recent modification: `Scaffolded` → `Modified` → `Validated` → `Deployed`. Any change after validation requires re-validation, preventing untested code deployment, a common failure mode when agents skip validation.

Agent compatibility. To validate agent-agnosticism, we tested the package across Claude Code (automated, primary) and Opencode with Sonnet. The key requirement is function calling capability; any agent that can invoke CLI tools works with our package.

3 Agentic Trajectory Analyzer

The analyzer consumes execution trajectories and recommends package improvements. This closes the loop: agents struggle → we see it in trajectories → we fix the tooling → agents struggle less.

End-state metrics (build success, test pass) don't reveal whether failures stem from model limitations, tool problems, missing domain knowledge, or underspecified prompts. Trajectories (the sequence of reasoning, tool calls, and results) show where things went wrong. An agent retrying the same malformed SQL five times reveals a missing example. An agent calling N tools for N tables reveals a missing batch operation.

Two-phase architecture. We employ a map-reduce approach optimized for cost and quality. In the *map phase*, each trajectory is processed independently by a fast model (Claude Haiku, ~ $0.001 per trajectory), extracting errors, retries, confusion patterns, and inefficient tool usage. This runs in parallel. In the *agentic synthesis phase*, aggregated patterns go to a reasoning model (Claude Opus) with read-only access to template source code, tool definitions, and prior analysis history, generating prioritized recommendations.

Cost model. For N trajectories: map costs $N \times$ ~$0.001 (fast model), synthesis costs ~$0.5–3 (reasoning model, bounded at 50 turns). For 50 apps, total analysis cost was under $15.

Human judgment on small N. On small datasets, we found that the analyzer can overreact, encountering an issue once or twice and recommending building a helper utility without assessing maintenance cost [4]. We recommend human review of recommendations when $N < 50$, treating the analyzer output as a prioritized error analysis rather than an implementation directive.

4 Agentic DevX Metrics

Agentic DevX is the final quality gate. After the package is improved and agents generate applications, we need to verify: can another agent actually operate this output?

Consider an agent that generates a working application. A human developer can run it: they'll figure out missing environment variables, install unlisted dependencies, work around unclear documentation. An agent *can sometimes* do this too, but it's slow and inefficient, spending many turns on trial-and-error that explicit configuration would eliminate.

Two evaluation modes. We employ two complementary approaches. *Shell-scripted* (**Ls**) metrics run deterministic commands—build tools, type checkers, test runners, HTTP probes—for reproducible pass/fail verdicts. *Agentic* (**Ra**) metrics deploy an evaluator agent (Claude Haiku) that receives a prompt contract (Table 1) and emits a structured `STATUS: PASS|FAIL` verdict, parsed via regex with fallback inference. Agentic metrics run three independent trials; a metric passes if at least two of three trials succeed. Each trial records turns and token usage.

Table 1: CanItShip metric definitions. Ls = shell-scripted, Ra = agentic.

ID	Metric	Check
Ls1	Build	Build scripts / import sanity
Ls2	Runtime	Start app, healthcheck on port
Ls3	Types	Static analysis (pyright, tsc)
Ls4	Tests	Run test suite; FAIL if none
Ls5	UI	Render page, check for errors
Ls6	Runability	Install, start, healthcheck
Ra7	Runability	Agent: install, start, verify
Ra8	Deployability	Validate bundle or config

The CanItShip framework. The eight metrics span three tiers: Core Functionality (Ls1–Ls4), Platform Integration (Ls5), and Agentic DevX (Ls6, Ra7–Ra8). Shell-scripted metrics provide deterministic, reproducible baselines; agentic metrics capture tasks (autonomous installation, deployment validation) that require adaptive tool use beyond fixed scripts. The naming convention encodes evaluation mode: **Ls** (Level, scripted) for deterministic checks and **Ra** (Readiness, agentic) for agent-evaluated checks. The harness moves toward platform independence: shell-scripted checks (Ls) require pre-built scripts per platform, but agentic checks (Ra) need only a

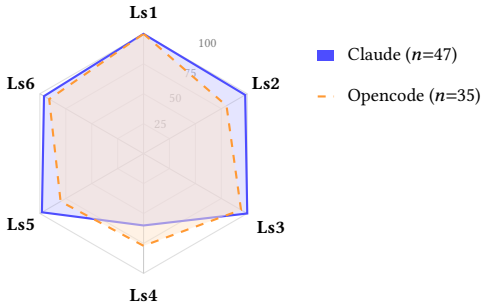


Figure 1: Shell-scripted metric profiles (Ls1–Ls6). Both agents achieve near-perfect build and type-check scores; the main gap is Ls4 (tests) and Ls5 (UI). Agentic metrics (Ra7–Ra8) evaluated on a subset are in Table 5.

Table 2: Dataset coverage. Set 1: 20 generic analytics prompts (churn, RFM, warehouse). Set 2: 30 Databricks-specific prompts (property search, bookings, parts). Generation failures produced only metadata with no source code.

	Gen.	Gen. fail	Ls eval	Ra eval
Claude (Set 1+2)	49/50	1	47	—
Opencode (Set 2)	28/30	2	22	—
Opencode (Set 1)	18/20	2	13	11

task prompt and a configured environment, making them portable by design.

Generation–evaluation isolation. Agents that can read evaluation scripts produce code that passes checks rather than solving the task—a form of reward hacking [2]. We enforce workspace-level isolation, excluding evaluation scripts from the generation environment. Each generation starts from a clean workspace, mirroring ML train/test splits.

5 Results

We organize results around three claims: (1) automated scripted evaluation on a new data-focused benchmark; (2) agentic checks revealing failures invisible to scripted evaluation; (3) a trajectory-driven closed loop that analyzes failures, applies fixes, and measures improvement.

The prior app.build evaluation [11] achieved 73.3% viability on 30 generic consumer prompts using human+cloud evaluation. We create 50 new data-focused prompts requiring SQL, schema access, and platform integration, and automate the evaluation as reproducible local shell scripts.

Claim 1: Automated scripted evaluation on new data-focused benchmark. Two disjoint prompt sets were given to both agents under identical conditions (Table 2). Shell-scripted checks (Ls1–Ls6) ran on both agents to measure how the harness interacts with each backend. Agentic checks (Ra7–Ra8) ran on Opencode Set 1 apps only; 9 were excluded because generation quality was already poor (no entry points, incomplete scaffolds).

Table 3: Shell-scripted evaluation (Ls metrics, n=47 Claude Code apps).

Tier	Metric	Result
Core	Ls1: Build Success	47/47 (100%)
	Ls2: Runtime Success	46/47 (98%)
	Ls3: Type Safety	47/47 (100%)
	Ls4: Tests Pass	28/47 (60%)
Platform	Ls5: UI Renders	46/47 (98%)
DevX	Ls6: Runability	45/47 (96%)
Efficiency	Cost per App	$1.61
	Turns per App	46 avg

Agentic metrics (Ra7–Ra8) in Table 5.

Shell-scripted metrics (Ls1–Ls6) were evaluated using deterministic build, runtime, type-check, test, UI, and HTTP probe commands with a 3-trial majority protocol for Ls6.

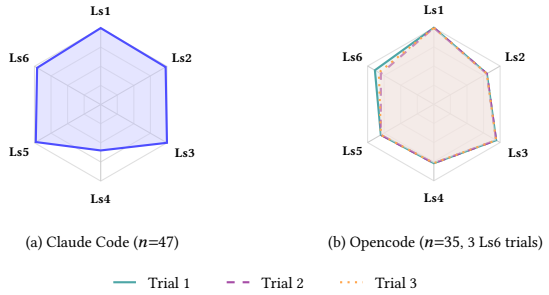
The 100% build and type-safety rates across all 47 apps confirm that Claude Code with installable domain knowledge produces structurally sound applications. Ls2 (runtime) and Ls5 (UI) each achieve 98%, with a single app failing to respond on the healthcheck endpoint. Ls6 (runability) reaches 96% under the 3-trial majority protocol. Ls4 (tests) shows 60%, as 19 apps' test suites contain integration tests that require a live database connection unavailable in local evaluation. Where app.build achieved 73.3% viability on generic consumer prompts with human evaluation, domain knowledge yields 100% build success on data-focused prompts with fully automated checks.

Multi-agent comparison. To validate agent-agnosticism, we ran the same shell-scripted harness on 35 Opencode+Sonnet applications across both prompt sets (Table 4). Both agents achieve 100% build (Ls1); divergence appears on runtime, UI, and DevX metrics. Claude Code's 98% runtime versus Opencode's 80% reflects architectural differences: Node.js apps (Claude) have complex startup sequences but consistent healthcheck patterns, while Python/Streamlit apps (Opencode) start faster but 7 apps fail the HTTP healthcheck due to CLI-only or non-standard entry points.

Table 4: Multi-agent shell-scripted (Ls) comparison: Claude (n=47) vs Opencode (n=35).

Metric	Claude	Opencode	Δ
Ls1: Build	100%	100%	0
Ls2: Runtime	98%	80%	+18
Ls3: Types	100%	94%	+6
Ls4: Tests	60%	77%	−17
Ls5: UI	98%	80%	+18
Ls6: Runability	96%	91%	+5

Claim 2: Agentic checks—can another agent operate these apps? Ra7 (agentic runability) and Ra8 (deployability) require adaptive tool use—autonomously installing and starting applications, and validating deployment configurations—beyond the scope of deterministic shell scripts. A key motivation is agent-agnostic coverage: when Opencode produced Python applications instead of



(a) Claude Code (n=47) (b) Opencode (n=35, 3 Ls6 trials)

Figure 2: Detailed metric profiles. (a) Claude Code (n=47): 100% build/types, 98% runtime/UI, 96% Ls6 runability. (b) Opencode (n=35): three Ls6 trial-level aggregates show variance (89%/80%/83%) from timing and port contention.

the Node.js stack our shell checks targeted, deterministic scripts lacked coverage for framework detection, dependency resolution, and startup sequences. An agentic evaluator handles this naturally by reading project files and adapting its strategy, mirroring how a human developer would onboard an unfamiliar codebase. This also makes the benchmark more faithful: when an evaluator agent struggles to run a formally correct application due to poor developer experience (missing entry points, undocumented setup steps), the resulting lower score reflects a genuine DevX deficiency rather than a tooling gap. We evaluated Ra7–Ra8 on a subset of n=11 Opencode+Sonnet applications using an evaluator agent (Claude Haiku) with three independent trials per metric and a 2/3 majority pass criterion (Table 5).

Table 5: Agentic evaluation (Ra metrics): Opencode+Sonnet subset (n=11, Claude Haiku evaluator, 3-trial 2/3 majority).

Metric	Pass	Rate	Failure Mode
Ra7: Runability	8/11	73%	Path escaping, startup
Ra8: Deployability	4/11	36%	Missing deploy config

Ra7 passed at 73%, lower than shell-scripted Ls6 (100% on the same apps), confirming that autonomous discovery is harder than scripted probes. Ra8 failures stemmed from apps lacking deployment configuration entirely.

Ls6 runability variance. The 3-trial shell-scripted protocol for Ls6 reveals non-determinism from timing and port contention. For Claude Code (n=47), 38 apps passed all 3 trials, 7 apps passed 2/3, and 2 apps passed 0/3, yielding 96% under the majority criterion. For Opencode (n=35), 24 apps passed all 3 trials, 8 apps passed 2/3, and 3 apps passed 0/3, yielding 91%. This variance—even in deterministic scripts—demonstrates why multi-trial protocols are essential for evaluation metrics involving process startup and network probes.

Claim 3: Trajectory analysis closes the loop. After agentic checks identify failing apps, the trajectory analyzer processes failure traces and recommends fixes. We apply fixes to the domain package and applications via Claude Code, then rerun agentic evaluations to measure improvement. The protocol is: (1) run agentic eval to identify failing apps and failure traces; (2) feed traces to the trajectory analyzer to obtain recommendations; (3) apply recommendations to the domain package and generated applications using Claude Code; (4) rerun agentic eval on the same apps and measure the delta.

Table 6 shows manual loop iterations: the trajectory analyzer identified friction points that we fixed in the domain package. These fixes—targeting tool descriptions, prompts, and examples—are amenable to automatic optimization [1, 9].

Table 6: Trajectory-identified improvements (manual loop iterations).

Pattern Found	Root Cause	Fix Applied
N calls for N tables	Missing batch op	Added batch schema discovery
Repeated SQL errors	Missing examples	Added QUALIFY, PIVOT to guidance
Confusing tool name	Name mismatch	Renamed tool for clarity
Retries on errors	Unclear messages	Added contextual error params
6% type safety	Missing null examples	Added null handling to templates

Table 7 shows the automated loop: applying analyzer recommendations from agentic evaluation failure traces and measuring improvement on the same apps.

Table 7: Trajectory-driven improvement: agentic metrics before and after applying analyzer recommendations. Results pending.

Metric	Before	After	Δ
Ra7: Runability	8/11 (73%)	–/– (–%)	–
Ra8: Deployability	4/11 (36%)	–/– (–%)	–

6 Related Work

Environment over model. Recent work [11] found that environment quality had larger effects than model upgrades when comparing agent performance. SWE-agent [17] similarly demonstrated that agent-computer interfaces significantly affect performance. This motivates our focus on improving tooling rather than agents.

Evaluation gap. Existing benchmarks evaluate code correctness (HumanEval [5], SWE-bench [8]), task completion (WebArena [19], GAIA [14]), or SQL quality (BIRD [12], Spider [18]). None ask whether generated code can be operated by other agents, a critical question for compound AI systems.

Trajectory analysis. Analyzing agent execution traces for debugging is established practice (SWE-agent [17], OpenHands [16]). Our contribution is making the synthesis phase itself agentic: an agent that explores source code to generate recommendations, rather than static aggregation.

Agent evaluation harnesses. Inspect AI [15] and DeepEval [7] provide evaluation infrastructure but focus on correctness rather than deployment readiness. AgentBench [13] spans multiple environments but does not evaluate autonomous deployability.

7 Discussion

Agent-agnostic vs agent-specific. Our approach invests in tooling rather than agents: agents improve rapidly (model upgrades are free to adopters), while domain knowledge is slow to encode and hard to transfer. The tradeoff is control; agent-specific approaches can fine-tune behavior precisely, while we rely on the agent's ability to follow tool descriptions. In practice, this suffices for structured tasks but limits open-ended architectural reasoning.

Toward automatic optimization. Our loop is currently semi-automatic: the analyzer outputs recommendations, a human reviews them. Since recommendations target tool descriptions and prompts, techniques like GEPA [1] and DSPy [9] could potentially close the loop without human intervention.

Limitations. *External validity:* We validate on Databricks; the harness is platform-independent but domain packages require adaptation. *Internal validity:* Shell metrics cover n=47 Claude and n=35 Opencode apps; agentic metrics cover n=11. Scaling to 100+ is planned. *Construct validity:* Even deterministic scripts exhibit timing variance (Figure 2); multi-trial protocols are essential. The feedback loop is demonstrated via case studies (Table 6); longitudinal quantification is planned.

8 Conclusion

We presented a feedback loop for iterative improvement of agent tooling: installable domain knowledge, an agentic trajectory analyzer, and CanItShip Agentic DevX metrics. Using shell-scripted (Ls) and agentic (Ra) evaluation across 82 Databricks applications and two agent backends (Claude Code n=47, Opencode+Sonnet n=35), we achieve 100% build and type safety, 98% runtime success, and 96% local runability at $1.61/app. Agentic evaluation on a subset (n=11) shows 73% agentic runability and 36% deployability, with failures traceable to missing deployment manifests. Beyond identifying friction through case studies, we demonstrate a closed-loop mechanism: the trajectory analyzer processes agentic evaluation failure traces, recommends fixes applied via Claude Code, and improvements are measured by rerunning the same evaluations. When agents fail, improve their environment rather than the agents themselves. The evaluation harness, trajectory analyzer, 50-prompt benchmark, and anonymized artifacts are released under the MIT license.

Acknowledgments

We thank the Databricks Apps team for infrastructure support.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2025. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv preprint arXiv:2507.19457* (2025).

[2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. 2016. Concrete Problems in AI Safety. *arXiv preprint arXiv:1606.06565* (2016).

[3] Anthropic. 2024. Tool Use with Claude. https://docs.anthropic.com/en/docs/build-with-claude/tool-use.

[4] Valerii Babushkin and Arseny Kravchenko. 2025. *Machine Learning System Design: With End-to-End Examples.* Manning Publications.

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Cloudflare. 2025. Code Mode: Give Agents an Entire API in 1,000 Tokens. https://blog.cloudflare.com/code-mode-mcp/.

[7] Confident AI. 2024. DeepEval: The LLM Evaluation Framework. https://github.com/confident-ai/deepeval.

[8] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770* (2024).

[9] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, et al. 2023. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. *arXiv preprint arXiv:2310.03714* (2023).

[10] Evgenii Kniazev. 2026. Automated Classification of Source Code Changes Based on Metrics Clustering in the Software Development Process. *arXiv preprint arXiv:2602.14591* (2026).

[11] Evgenii Kniazev, Arseny Kravchenko, Igor Rekun, James Broadhead, Nikita Shamgunov, Pranav Sah, Pratik Nichite, and Ivan Yamshchikov. 2025. app.build: A Production Framework for Scaling Agentic Prompt-to-App Generation with Environment Scaffolding. arXiv:2509.03310 [cs.AI]

[12] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL. *Advances in Neural Information Processing Systems* 36 (2023).

[13] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. AgentBench: Evaluating LLMs as Agents. *arXiv preprint arXiv:2308.03688* (2023).

[14] Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raez, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. 2023. GAIA: A Benchmark for General AI Assistants. *arXiv preprint arXiv:2311.12983* (2023).

[15] UK AI Safety Institute. 2024. Inspect: A Framework for Large Language Model Evaluations. https://inspect.aisi.org.uk/.

[16] Xingyao Wang, Boxuan Tian, Jiahui Chen, Bowen Li, Ziyi Peng, Tianjun Shi, Robert Hurst, Zijian Yi, Sanjay Jain, Bhavya Vontimitta, et al. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741* (2024).

[17] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Liber, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793* (2024).

[18] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *arXiv preprint arXiv:1809.08887* (2018).

[19] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854* (2024).